

The Monotone Forward–Backward Algorithm and Its Gradients

A derivation for the ND-Aligner CUDA extension

Suseong Kim

1 Introduction

This note derives the monotone forward–backward recursions [2, 10] and their gradients in the form in which they are implemented as a CUDA extension for ND-Aligner [8], so that the kernels can be read against an independent statement of what they compute rather than against themselves. The implementation itself is out of scope: indices are zero-based to match it, and Appendix E treats the one place where a choice between equivalent formulas is made on numerical grounds, but the parallelization and memory layout are not discussed.

The gradients are derived for an arbitrary downstream computation rather than for a fixed objective. The reason is that the posterior marginals are themselves an intermediate quantity: they are consumed by losses defined directly on the alignment, and by a reconstruction path that expands text representations through them, so gradient arrives at the forward and the backward variables at once. Were $-\log Z$ the only objective, the marginals would already be its gradient and no derivation would be needed; it is the general case that requires the two branches of Section 3.

Section 2 develops the algorithm. It sets up alignment as inference over a space of monotone paths, defines the forward and backward variables as sums over path prefixes and suffixes, derives the recursions that evaluate those sums in $O(TN)$ time in place of a combinatorially large enumeration, and combines the two into the posterior marginal of each frame–token pair. Section 3 differentiates the recursions with respect to the potentials they consume. The derivative splits into two branches of different shapes: the forward branch reduces to a single adjoint array, the backward branch to two arrays that must advance together. Both cost $O(TN)$, and both redistribute gradient along the conditional transition distributions that the passes they differentiate used to accumulate mass.

A reader who wants only the formulas can take the forward recursion from (11) with (8), the backward recursion from (18) with (14), and the marginals from Proposition 1. On the gradient side, the alpha branch is (60) started from (58) with the first-frame exception (64); the beta branch is the pair (86) started from (87); and the weights are (61) and (89). Proposition 12 gives $\partial \log Z / \partial \hat{\phi}(t, j) = \gamma_{t,j}$, which ties the two halves together and serves as a self-check on an implementation of both.

Proofs that verify rather than derive are collected in the appendices, along with the chain rule in the form used throughout (Appendix A) and the stable evaluation of LSE (Appendix E).

2 The forward–backward algorithm

2.1 Setup

Let T be the number of speech frames and N the number of text tokens, with $T \geq N$. Indices are zero-based throughout, matching the implementation: $t \in \{0, \dots, T-1\}$ and $j \in \{0, \dots, N-1\}$.

An alignment is a path $z = (z_0, \dots, z_{T-1})$ that assigns frame t to token z_t . Let $\mathcal{S} \subseteq \{0, \dots, N-1\}$ be the set of *optional* tokens, those a path is permitted to pass over without spending a frame on them. A path is *admissible* when its increments satisfy

$$z_{t+1} - z_t \in \begin{cases} \{0, 1, 2\}, & z_t + 1 \in \mathcal{S}, \\ \{0, 1\}, & z_t + 1 \notin \mathcal{S}, \end{cases} \quad 0 \leq t \leq T-2, \quad (1)$$

subject to $z_{t+1} \leq N-1$. The three increments are *stay*, *advance*, and *optional skip*; the skip is available only when the token being passed over is itself optional. The admissible start states are

$$z_0 \in \begin{cases} \{0, 1\}, & 0 \in \mathcal{S}, \\ \{0\}, & 0 \notin \mathcal{S}, \end{cases} \quad (2)$$

and every path must end at $z_{T-1} = N-1$. The monotone path space $\mathcal{P}_M$ is the set of paths satisfying (1), (2), and this terminal condition.

Each frame–token pair carries a nonnegative node potential $\phi(t, j)$, whose logarithm we write as

$$\hat{\phi}(t, j) = \log \phi(t, j). \quad (3)$$

A path is scored by the product of the potentials it visits,

$$\Psi(z) = \prod_{t=0}^{T-1} \phi(t, z_t), \quad \log \Psi(z) = \sum_{t=0}^{T-1} \hat{\phi}(t, z_t). \quad (4)$$

There is no transition potential: monotonicity is imposed through the support $\mathcal{P}_M$ rather than through a score attached to transitions. The construction is that of a conditional random field [9, 12] whose chain structure has been moved out of the potentials and into the support, and the resulting sum over monotone paths is the quantity that forward-sum alignment objectives maximize [5, 11, 1]. The quantity of interest is the partition function over monotone paths,

$$Z = \sum_{z \in \mathcal{P}_M} \Psi(z). \quad (5)$$

The number of paths in $\mathcal{P}_M$ grows combinatorially in T and N ; the recursions below evaluate (5) in $O(TN)$ time.

All quantities are computed in the log domain. We write $\hat{\alpha}_{t,j} \equiv \log \alpha_{t,j}$ and $\hat{\beta}_{t,j} \equiv \log \beta_{t,j}$ for the forward and backward variables defined below, and for a finite list of candidates $x_1, \dots, x_m$,

$$\text{LSE}(x_1, \dots, x_m) = \log \sum_{r=1}^m e^{x_r}. \quad (6)$$

States that are invalid or unreachable are assigned a large negative constant standing in for $-\infty$.

2.2 Forward recursion

2.2.1 Definition

Call $(z_0, \dots, z_t)$ an *admissible prefix ending at k* when it satisfies (1) for $0 \leq s \leq t-1$, satisfies (2), and has $z_t = k$. Write the set of such prefixes as $\mathcal{A}_{t,k}$, and define

$$\hat{\alpha}_{t,k} = \log \sum_{(z_0, \dots, z_t) \in \mathcal{A}_{t,k}} \exp \left(\sum_{s=0}^t \hat{\phi}(s, z_s) \right). \quad (7)$$

Two features of (7) are used repeatedly. First, the inner sum runs up to and including $s = t$, so $\hat{\alpha}_{t,k}$ carries the potential $\hat{\phi}(t, k)$ of its own endpoint. Second, the terminal condition $z_{T-1} = N-1$ plays no role here, since a prefix is unconstrained to the right.

2.2.2 Initialization

At $t = 0$ a prefix is the single state (z_0) , and (7) reduces to the potential of that state whenever it is an admissible start. By (2),

$$\hat{\alpha}_{0,k} = \begin{cases} \hat{\phi}(0, 0), & k = 0, \\ \hat{\phi}(0, 1), & k = 1 \text{ and } 0 \in \mathcal{S}, \\ -\infty, & \text{otherwise.} \end{cases} \quad (8)$$

The second case is the optional start: when token 0 may be skipped, a path is allowed to begin on token 1.

2.2.3 Recursion

Fix $t \geq 1$ and k . Every prefix in $\mathcal{A}_{t,k}$ arrives at k from exactly one predecessor $k' \in \{k, k-1, k-2\}$, where by (1) the predecessor $k-2$ is admissible only when the token passed over, namely $k-1$, is optional. Deleting the final state of a prefix in $\mathcal{A}_{t,k}$ yields a prefix in $\mathcal{A}_{t-1,k'}$ for exactly one such k' , and conversely appending k to any prefix in $\mathcal{A}_{t-1,k'}$ yields a prefix in $\mathcal{A}_{t,k}$. These two operations are mutually inverse, so

$$\mathcal{A}_{t,k} = (\mathcal{A}_{t-1,k} \oplus k) \sqcup (\mathcal{A}_{t-1,k-1} \oplus k) \sqcup [k-1 \in \mathcal{S}] (\mathcal{A}_{t-1,k-2} \oplus k), \quad (9)$$

where $\cdot \oplus k$ denotes appending the state k to every prefix in a set, the union is disjoint because the three families differ in their second-to-last state, and the bracket is 1 when the condition holds and 0 otherwise.

Each appended state contributes the same factor $e^{\hat{\phi}(t,k)}$ to the weight, so substituting (9) into (7) and pulling that factor out of every term gives

$$e^{\hat{\alpha}_{t,k}} = e^{\hat{\phi}(t,k)} \left(e^{\hat{\alpha}_{t-1,k}} + e^{\hat{\alpha}_{t-1,k-1}} + [k-1 \in \mathcal{S}] e^{\hat{\alpha}_{t-1,k-2}} \right). \quad (10)$$

Taking logarithms of (10) and enumerating the cases in which each predecessor lies in range,

$$\hat{\alpha}_{t,k} = \hat{\phi}(t, k) + \begin{cases} \text{LSE}(\hat{\alpha}_{t-1,k}, \hat{\alpha}_{t-1,k-1}, \hat{\alpha}_{t-1,k-2}), & k \geq 2, \ k-1 \in \mathcal{S}, \\ \text{LSE}(\hat{\alpha}_{t-1,k}, \hat{\alpha}_{t-1,k-1}), & k \geq 2, \ k-1 \notin \mathcal{S}, \\ \text{LSE}(\hat{\alpha}_{t-1,1}, \hat{\alpha}_{t-1,0}), & k = 1, \\ \hat{\alpha}_{t-1,0}, & k = 0. \end{cases} \quad (11)$$

At $k = 1$ no skip is available, since the predecessor $k - 2 = -1$ is out of range, and at $k = 0$ only a stay remains.

2.2.4 Partition function

Every path in $\mathcal{P}_M$ ends at $z_{T-1} = N - 1$, and $\mathcal{A}_{T-1, N-1}$ is exactly the set of admissible paths ending there, that is, $\mathcal{P}_M$ itself. Evaluating (7) at $(t, k) = (T - 1, N - 1)$ and comparing with (4) and (5),

$$\hat{\alpha}_{T-1, N-1} = \log \sum_{z \in \mathcal{P}_M} \exp \left(\sum_{s=0}^{T-1} \hat{\phi}(s, z_s) \right) = \log \sum_{z \in \mathcal{P}_M} \Psi(z) = \log Z. \quad (12)$$

2.3 Backward recursion

2.3.1 Definition

Call $(z_t, \dots, z_{T-1})$ an *admissible suffix starting at j* when it satisfies (1) for $t \leq s \leq T - 2$, has $z_t = j$, and ends at $z_{T-1} = N - 1$. Write the set of such suffixes as $\mathcal{B}_{t,j}$, and define

$$\hat{\beta}_{t,j} = \log \sum_{(z_t, \dots, z_{T-1}) \in \mathcal{B}_{t,j}} \exp \left(\sum_{s=t+1}^{T-1} \hat{\phi}(s, z_s) \right). \quad (13)$$

The inner sum begins at $s = t + 1$, so $\hat{\beta}_{t,j}$ *excludes* the potential $\hat{\phi}(t, j)$ of its own starting state. This is a convention rather than a necessity; Section 2.4 shows why it is the convenient one. Symmetrically to the forward case, the start condition (2) plays no role here, since a suffix is unconstrained to the left.

2.3.2 Initialization

At $t = T - 1$ a suffix is the single state (z_{T-1}) , which must equal $N - 1$. The inner sum in (13) is then empty and evaluates to zero, so

$$\hat{\beta}_{T-1,j} = \begin{cases} 0, & j = N - 1, \\ -\infty, & j \neq N - 1. \end{cases} \quad (14)$$

2.3.3 Recursion

Fix $t \leq T - 2$ and j . Every suffix in $\mathcal{B}_{t,j}$ leaves j for exactly one successor $k \in \{j, j + 1, j + 2\}$, with $k = j + 2$ admissible only when the token passed over, namely $j + 1$, is optional. Deleting the first state of a suffix in $\mathcal{B}_{t,j}$ yields a suffix in $\mathcal{B}_{t+1,k}$, and conversely prepending j to a suffix in $\mathcal{B}_{t+1,k}$ yields a suffix in $\mathcal{B}_{t,j}$. These operations are mutually inverse, so

$$\mathcal{B}_{t,j} = (j \oplus \mathcal{B}_{t+1,j}) \sqcup (j \oplus \mathcal{B}_{t+1,j+1}) \sqcup [j + 1 \in \mathcal{S}] (j \oplus \mathcal{B}_{t+1,j+2}), \quad (15)$$

with the union disjoint because the three families differ in their second state.

Unlike the forward case, the prepended state j contributes nothing to the log-weight in (13), because that sum begins at $s = t + 1$. What does appear is the potential of the state entered at $t + 1$: a suffix $j \oplus (z_{t+1}, \dots, z_{T-1})$ with $z_{t+1} = k$ has log-weight

$$\sum_{s=t+1}^{T-1} \hat{\phi}(s, z_s) = \hat{\phi}(t+1, k) + \sum_{s=t+2}^{T-1} \hat{\phi}(s, z_s), \quad (16)$$

whose second term is precisely what $\hat{\beta}_{t+1,k}$ accumulates. Substituting (15) into (13) and applying (16) to each family therefore gives

$$e^{\hat{\beta}_{t,j}} = e^{\hat{\phi}(t+1,j) + \hat{\beta}_{t+1,j}} + e^{\hat{\phi}(t+1,j+1) + \hat{\beta}_{t+1,j+1}} + [j+1 \in \mathcal{S}] e^{\hat{\phi}(t+1,j+2) + \hat{\beta}_{t+1,j+2}}. \quad (17)$$

Taking logarithms of (17) and enumerating the cases in which each successor lies in range,

$$\hat{\beta}_{t,j} = \begin{cases} \text{LSE}\left(\hat{\phi}(t+1, j) + \hat{\beta}_{t+1,j}, \hat{\phi}(t+1, j+1) + \hat{\beta}_{t+1,j+1}, \right. \\ \quad \left. \hat{\phi}(t+1, j+2) + \hat{\beta}_{t+1,j+2}\right), & j \leq N-3, \quad j+1 \in \mathcal{S}, \\ \text{LSE}\left(\hat{\phi}(t+1, j) + \hat{\beta}_{t+1,j}, \hat{\phi}(t+1, j+1) + \hat{\beta}_{t+1,j+1}\right), & j \leq N-3, \quad j+1 \notin \mathcal{S}, \\ \text{LSE}\left(\hat{\phi}(t+1, N-2) + \hat{\beta}_{t+1,N-2}, \hat{\phi}(t+1, N-1) + \hat{\beta}_{t+1,N-1}\right), & j = N-2, \\ \hat{\phi}(t+1, N-1) + \hat{\beta}_{t+1,N-1}, & j = N-1. \end{cases} \quad (18)$$

At $j = N - 2$ the skip would land on token N , out of range, and at $j = N - 1$ only a stay remains.

2.3.4 Partition function

By (2) the admissible start states are $z_0 = 0$ and, when $0 \in \mathcal{S}$, also $z_0 = 1$. Every path in $\mathcal{P}_M$ is an admissible suffix starting at one of these, so $\mathcal{P}_M = \mathcal{B}_{0,0} \sqcup [0 \in \mathcal{S}] \mathcal{B}_{0,1}$ and

$$Z = \sum_{z \in \mathcal{B}_{0,0}} \exp\left(\sum_{s=0}^{T-1} \hat{\phi}(s, z_s)\right) + [0 \in \mathcal{S}] \sum_{z \in \mathcal{B}_{0,1}} \exp\left(\sum_{s=0}^{T-1} \hat{\phi}(s, z_s)\right). \quad (19)$$

Splitting off the $s = 0$ term of each log-weight, which equals $\hat{\phi}(0, z_0)$ and is constant within each family, and applying (13) to the remainder,

$$Z = e^{\hat{\phi}(0,0)} e^{\hat{\beta}_{0,0}} + [0 \in \mathcal{S}] e^{\hat{\phi}(0,1)} e^{\hat{\beta}_{0,1}}, \quad (20)$$

that is,

$$\log Z = \begin{cases} \text{LSE}(\hat{\phi}(0,0) + \hat{\beta}_{0,0}, \hat{\phi}(0,1) + \hat{\beta}_{0,1}), & 0 \in \mathcal{S}, \\ \hat{\phi}(0,0) + \hat{\beta}_{0,0}, & 0 \notin \mathcal{S}. \end{cases} \quad (21)$$

Equations (12) and (21) are two expressions for the same quantity. The two recursions traverse the grid in opposite directions and share no intermediate state, so their agreement is a useful invariant: it fails whenever either recursion, either initialization, or the handling of optional tokens is wrong in a way that changes the total mass.

2.4 Posterior marginals

Let p be the distribution over monotone paths induced by the potentials,

$$p(z) = \frac{\Psi(z)}{Z}, \quad z \in \mathcal{P}_M, \quad (22)$$

which is normalized by the definition of Z in (5). The *posterior marginal* is the probability that frame t is assigned to token j ,

$$\gamma_{t,j} = \Pr_{z \sim p}[z_t = j] = \frac{1}{Z} \sum_{z \in \mathcal{P}_M^{t,j}} \Psi(z), \quad \mathcal{P}_M^{t,j} \equiv \{z \in \mathcal{P}_M : z_t = j\}. \quad (23)$$

Proposition 1. *For every t and j ,*

$$\gamma_{t,j} = \exp(\hat{\alpha}_{t,j} + \hat{\beta}_{t,j} - \log Z). \quad (24)$$

Proof sketch. Cutting a path with $z_t = j$ at frame t into its prefix $(z_0, \dots, z_t)$ and suffix $(z_t, \dots, z_{T-1})$, which share the state j , is a bijection onto $\mathcal{A}_{t,j} \times \mathcal{B}_{t,j}$: admissibility (1) constrains only consecutive states, so any prefix may be joined to any suffix at their common endpoint. The log-weight splits along the same cut, with $\hat{\phi}(t, j)$ falling on the prefix side and nowhere else, which is the exclusion built into (13). Summing over the product set factorizes into $e^{\hat{\alpha}_{t,j}} e^{\hat{\beta}_{t,j}}$, and dividing by Z gives the claim. Appendix B gives the details. $\square$

The convention adopted in (13) is what makes (106) an identity. Had $\hat{\beta}_{t,j}$ been defined with its own starting potential included, the split (101) would carry $\hat{\phi}(t, j)$ on both sides, the product in (106) would read $e^{\hat{\alpha}_{t,j} + \hat{\beta}_{t,j} + \hat{\phi}(t,j)}$, and recovering the marginal would require dividing out $\phi(t, j)$ at every cell of the grid.

Proposition 2. *For every t ,*

$$\sum_{j=0}^{N-1} \gamma_{t,j} = 1. \quad (25)$$

Proof sketch. Every path assigns frame t to exactly one token, so the sets $\mathcal{P}_M^{t,0}, \dots, \mathcal{P}_M^{t,N-1}$ partition $\mathcal{P}_M$. Summing (106) over j therefore recovers Z ; dividing by it gives the claim. Appendix B gives the details. $\square$

Proposition 2 rests only on (108), which holds for any admissible transition structure, the optional skip included: it depends on each path occupying exactly one state per frame, not on the particular set of increments in (1). Consequently $\gamma_{t,\cdot}$ is a distribution over tokens for every frame, and $\gamma \in \mathbb{R}^{T \times N}$ is row-stochastic. Cells that no monotone path can reach receive $\gamma_{t,j} = 0$, since $\mathcal{P}_M^{t,j}$ is then empty and (106) gives $e^{\hat{\alpha}_{t,j} + \hat{\beta}_{t,j}} = 0$; the row sums are therefore unaffected by masking such cells.

3 Differentiating the recursions

3.1 The objective as a composition

The two recursions take the same input. Both read the array $\hat{\phi} \in \mathbb{R}^{T \times N}$, the forward recursion (11) referring in addition only to forward variables at earlier frames and the backward recursion

(18) only to backward variables at later frames, so neither consumes the output of the other. Write the two maps they define as

$$\hat{\alpha} = \mathcal{F}(\hat{\phi}), \quad \hat{\beta} = \mathcal{G}(\hat{\phi}). \quad (26)$$

Everything computed afterwards — the partition function of (5), the posterior marginals of (23), and whatever objective is built on them — reaches $\hat{\phi}$ only through this pair. Calling that computation $\mathcal{L}$, a function of two array arguments, the objective is the composition

$$L(\hat{\phi}) = \mathcal{L}(\mathcal{F}(\hat{\phi}), \mathcal{G}(\hat{\phi})). \quad (27)$$

What $\mathcal{L}$ supplies is the pair of partial derivatives $\partial\mathcal{L}/\partial\hat{\alpha}_{t,j}$ and $\partial\mathcal{L}/\partial\hat{\beta}_{t,j}$, each taken with the other argument of $\mathcal{L}$ held fixed; in reverse-mode automatic differentiation [6, 3] these are the *upstream gradients* handed to the backward pass. What is to be produced is the derivative of the composition,

$$\frac{\partial L}{\partial\hat{\phi}(t,j)}, \quad 0 \leq t \leq T-1, \quad 0 \leq j \leq N-1. \quad (28)$$

The two are different derivatives of different functions, which is why $\mathcal{L}$ and L are distinguished: the former treats $\hat{\alpha}$ and $\hat{\beta}$ as free arguments, the latter accounts for their dependence on $\hat{\phi}$.

In (27) the array $\hat{\phi}$ is a shared input to two computations that are otherwise independent, and $\{\hat{\alpha}_{t,j}\} \cup \{\hat{\beta}_{t,j}\}$ is the complete set of intermediate quantities through which $\hat{\phi}$ reaches L . The chain rule in the form of Lemma 13, stated in Appendix A and used throughout what follows, therefore splits the derivative into two sums, one term for each intermediate component:

$$\frac{\partial L}{\partial\hat{\phi}(t,j)} = \underbrace{\sum_{t'=0}^{T-1} \sum_{k=0}^{N-1} \frac{\partial\mathcal{L}}{\partial\hat{\alpha}_{t',k}} \frac{\partial\hat{\alpha}_{t',k}}{\partial\hat{\phi}(t,j)}}_{g_{t,j}^{(\alpha)}} + \underbrace{\sum_{t'=0}^{T-1} \sum_{k=0}^{N-1} \frac{\partial\mathcal{L}}{\partial\hat{\beta}_{t',k}} \frac{\partial\hat{\beta}_{t',k}}{\partial\hat{\phi}(t,j)}}_{g_{t,j}^{(\beta)}}, \quad (29)$$

in which each right-hand factor of the form $\partial\hat{\alpha}_{t',k}/\partial\hat{\phi}(t,j)$ is itself a derivative through the recursion (11), and likewise for $\hat{\beta}$ through (18). We call $g^{(\alpha)}$ and $g^{(\beta)}$ the *alpha branch* and *beta branch* gradients. Since their sum is the required result, it suffices to compute the two separately and add them; Sections 3.3 and 3.4 do so.

Neither sum in (29) is evaluated as written. A single $\hat{\phi}(t,j)$ influences $\Theta(TN)$ of the intermediate variables, so forming the sums directly would cost $\Theta(T^2N^2)$ for the full gradient array. Both branches instead admit a recursion over frames, at the same $O(TN)$ cost as the forward and backward passes themselves.

3.2 The derivative of a log-sum-exp

Both recursions are assembled from LSE, so its derivative is the only local rule needed. Differentiating the definition, for candidates $x_1, \dots, x_m$,

$$\frac{\partial}{\partial x_i} \text{LSE}(x_1, \dots, x_m) = \frac{e^{x_i}}{\sum_{r=1}^m e^{x_r}} = \text{softmax}_i(x_1, \dots, x_m). \quad (30)$$

The derivative with respect to one candidate is thus the softmax weight of that candidate within the list. Being a softmax, these weights are nonnegative and sum to one,

$$\text{softmax}_i(x_1, \dots, x_m) \geq 0, \quad \sum_{i=1}^m \text{softmax}_i(x_1, \dots, x_m) = 1. \quad (31)$$

Two consequences recur below. First, by (31) every local derivative appearing in the two branches is a convex weight, so the recursions redistribute gradient among states without inflating or discarding it. Second, in both recursions the candidates passed to LSE are the contributions of the admissible transitions into or out of a state, so the resulting weights are not merely algebraic: each is the conditional probability of taking one of those transitions. Sections 3.3.4 and 3.4.4 make this precise.

Appendix E gives the max-shifted form of (30) used in the implementation, and explains why the weights are recomputed from the candidate list rather than reconstructed from a stored recursion output.

3.3 Alpha branch

This section evaluates the first sum of (29),

$$g_{t,j}^{(\alpha)} = \sum_{t'=0}^{T-1} \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t',k}} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\phi}(t,j)}. \quad (32)$$

Most of the work is in the inner factors $\partial \hat{\alpha}_{t',k} / \partial \hat{\phi}(t,j)$, which are derivatives through the recursion (11) and are not available directly. We first establish which of them vanish and how the surviving ones factor, then use that structure to replace (32) by a recursion.

3.3.1 Dependency structure of the forward recursion

Lemma 3. $\hat{\alpha}_{t',k}$ is a function of $\hat{\phi}(s, \cdot)$ for $s \leq t'$ only. Consequently

$$\frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\phi}(t,j)} = 0 \quad \text{whenever } t' < t. \quad (33)$$

Proof. Induction on t' . At $t' = 0$ the initialization (8) involves only $\hat{\phi}(0, \cdot)$. For $t' \geq 1$ the recursion (11) forms $\hat{\alpha}_{t',k}$ from $\hat{\phi}(t', k)$ and from forward variables at frame $t' - 1$, each of which is by hypothesis a function of $\hat{\phi}(s, \cdot)$ for $s \leq t' - 1$. Hence $\hat{\alpha}_{t',k}$ involves only $\hat{\phi}(s, \cdot)$ for $s \leq t'$, and a variable it does not involve has zero derivative. $\square$

It is convenient to name the candidate set that (11) passes to LSE. Collecting the four cases of (11), the *admissible predecessors* of a token k are

$$\mathcal{K}(k) = \begin{cases} \{k, k-1, k-2\}, & k \geq 2, \ k-1 \in \mathcal{S}, \\ \{k, k-1\}, & k \geq 2, \ k-1 \notin \mathcal{S}, \\ \{1, 0\}, & k = 1, \\ \{0\}, & k = 0, \end{cases} \quad (34)$$

so that the recursion reads

$$\hat{\alpha}_{s,k} = \hat{\phi}(s, k) + \text{LSE}_{j \in \mathcal{K}(k)}(\hat{\alpha}_{s-1,j}), \quad s \geq 1, \quad (35)$$

where $\text{LSE}_{j \in \mathcal{K}(k)}$ denotes LSE applied to the candidates $\hat{\alpha}_{s-1,j}$ for j ranging over $\mathcal{K}(k)$.

Read (35) as a map that advances one frame. Writing $\hat{\alpha}_{s,\cdot} \in \mathbb{R}^N$ for the vector of forward variables at frame s , define $R_s : \mathbb{R}^N \rightarrow \mathbb{R}^N$ by

$$(R_s(v))_k = \hat{\phi}(s, k) + \text{LSE}_{j \in \mathcal{K}(k)}(v_j), \quad v \in \mathbb{R}^N, \quad (36)$$

so that (35) becomes

$$\hat{\alpha}_{s,\cdot} = R_s(\hat{\alpha}_{s-1,\cdot}), \quad s \geq 1. \quad (37)$$

The only ingredients of R_s besides v are the potentials $\hat{\phi}(s, \cdot)$, which are inputs of the whole computation and therefore constants. Because (37) reads frame $s - 1$ and no earlier frame, the maps compose: applying it repeatedly from t' downwards,

$$\hat{\alpha}_{t',\cdot} = R_{t'}(\hat{\alpha}_{t'-1,\cdot}) \quad (38)$$

$$= R_{t'}(R_{t'-1}(\hat{\alpha}_{t'-2,\cdot})) \quad (39)$$

$\vdots$

$$= (R_{t'} \circ R_{t'-1} \circ \cdots \circ R_{\tau+1})(\hat{\alpha}_{\tau,\cdot}) \equiv C_{\tau \rightarrow t'}(\hat{\alpha}_{\tau,\cdot}), \quad (40)$$

the composition stopping as soon as frame τ is reached and being the identity when $\tau = t'$. Componentwise,

$$\hat{\alpha}_{t',k} = (C_{\tau \rightarrow t'}(\hat{\alpha}_{\tau,\cdot}))_k, \quad \tau \leq t', \quad (41)$$

which exhibits $\hat{\alpha}_{t',k}$ as an explicit function of the N numbers $\hat{\alpha}_{\tau,0}, \dots, \hat{\alpha}_{\tau,N-1}$ and of constants. Two chain rules follow.

Lemma 4. For $t \leq \tau \leq t'$,

$$\frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t,j}} = \sum_{m=0}^{N-1} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{\tau,m}} \frac{\partial \hat{\alpha}_{\tau,m}}{\partial \hat{\alpha}_{t,j}}. \quad (42)$$

Proof. By (41), $\hat{\alpha}_{t',k}$ is a function of the components $\hat{\alpha}_{\tau,m}$ and of constants, the latter being the potentials $\hat{\phi}(s, \cdot)$ with $\tau < s \leq t'$. Since $t \leq \tau$, the variable $\hat{\alpha}_{t,j}$ can act on $\hat{\alpha}_{t',k}$ only through those components. This is the situation of Lemma 13 with N inner maps sharing the argument $\hat{\alpha}_{t,j}$, one for each m , and the chain rule gives (42). $\square$

Lemma 4 is where the one-frame reach of (35) is spent. Had the recursion read two frames back, say $\hat{\alpha}_{s,\cdot} = R_s(\hat{\alpha}_{s-1,\cdot}, \hat{\alpha}_{s-2,\cdot})$, the maps would not compose as in (40), $\hat{\alpha}_{t,j}$ would reach $\hat{\alpha}_{t',k}$ without passing through frame τ , and (42) would be false.

The second chain rule is the same statement with a potential in place of a forward variable.

Lemma 5. For $t \leq t'$ with $t \geq 1$,

$$\frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\phi}(t,j)} = \sum_{m=0}^{N-1} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t,m}} \frac{\partial \hat{\alpha}_{t,m}}{\partial \hat{\phi}(t,j)}. \quad (43)$$

Proof. Take $\tau = t$ in (41). The constants of $C_{t \rightarrow t'}$ are the potentials $\hat{\phi}(s, \cdot)$ with $s > t$, so $\hat{\phi}(t, j)$ can act on $\hat{\alpha}_{t',k}$ only through the components $\hat{\alpha}_{t,m}$. This is again the situation of Lemma 13 with N inner maps sharing one argument, and the chain rule gives (43). $\square$

The sums in (42) and (43) run over all N tokens rather than over $\mathcal{K}$, for the reason given after Lemma 13: both rest only on the layered structure (41), and a component the composition ignores contributes a vanishing derivative rather than being excluded in advance. It is Section 3.3.4 that puts $\mathcal{K}$ to work.

The inner factors of (43) are settled by the last of these structural properties: within its own frame, a potential reaches exactly one forward variable.

Lemma 6. *For $t \geq 1$,*

$$\frac{\partial \hat{\alpha}_{t,m}}{\partial \hat{\phi}(t,j)} = \delta_{mj} = \begin{cases} 1, & m = j, \\ 0, & m \neq j. \end{cases} \quad (44)$$

Proof. By (35),

$$\hat{\alpha}_{t,m} = \hat{\phi}(t, m) + \text{LSE}_{j' \in \mathcal{K}(m)}(\hat{\alpha}_{t-1, j'}), \quad (45)$$

in which $\hat{\phi}(t, j)$ can occur only in the first term, and there only when $m = j$. It cannot occur in the second, whose arguments $\hat{\alpha}_{t-1, \cdot}$ involve the potentials of frames $s \leq t-1$ alone by Lemma 3. Differentiating the first term therefore gives 1 when $m = j$ and 0 otherwise. $\square$

3.3.2 Reduction to an adjoint

Fix $t \geq 1$ and j . Starting from the definition (32),

$$g_{t,j}^{(\alpha)} = \sum_{t'=0}^{T-1} \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t',k}} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\phi}(t,j)} \quad (46)$$

$$= \sum_{t'=t}^{T-1} \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t',k}} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\phi}(t,j)} \quad \text{by Lemma 3} \quad (47)$$

$$= \sum_{t'=t}^{T-1} \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t',k}} \sum_{m=0}^{N-1} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t,m}} \frac{\partial \hat{\alpha}_{t,m}}{\partial \hat{\phi}(t,j)} \quad \text{by Lemma 5} \quad (48)$$

$$= \sum_{t'=t}^{T-1} \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t',k}} \sum_{m=0}^{N-1} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t,m}} \delta_{mj} \quad \text{by Lemma 6} \quad (49)$$

$$= \sum_{t'=t}^{T-1} \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t',k}} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t,j}} \equiv D_{t,j}^{(\alpha)}. \quad (50)$$

Line (47) drops the terms in which the potential lies in the future of the forward variable. Line (48) routes the remaining derivatives through frame t , and line (49) collapses that routing to its single surviving term, since the potential $\hat{\phi}(t, j)$ affects only the state j within its own frame. The last line defines the *alpha adjoint* $D_{t,j}^{(\alpha)}$, and states that the branch gradient at a potential coincides with the adjoint of the forward variable that potential is added to.

Frame $t = 0$ is exceptional, since (8) rather than (35) governs it and (44) was derived from the latter; Section 3.3.4 returns to it.

The problem is now to compute $D^{(\alpha)}$, and (50) is no easier to evaluate as written than (32) was. The next subsection turns it into a recursion.

3.3.3 The adjoint recursion

Split the outer sum of (50) at its first term. By (37) the components of one frame are each computed from the previous frame and not from one another, so within a frame they are independent variables and

$$\frac{\partial \hat{\alpha}_{t,k}}{\partial \hat{\alpha}_{t,j}} = \delta_{kj} = \begin{cases} 1, & k = j, \\ 0, & k \neq j. \end{cases} \quad (51)$$

The $t' = t$ term of (50) therefore collapses to a single entry of the upstream array,

$$\sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t,k}} \frac{\partial \hat{\alpha}_{t,k}}{\partial \hat{\alpha}_{t,j}} = \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t,k}} \delta_{kj} = \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t,j}}, \quad (52)$$

the last step keeping only the term with $k = j$. Hence

$$D_{t,j}^{(\alpha)} = \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t,j}} + \sum_{t'=t+1}^{T-1} \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t',k}} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t,j}}. \quad (53)$$

Every remaining term has $t' \geq t+1$, so Lemma 4 applies with $\tau = t+1$:

$$\frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t,j}} = \sum_{m=0}^{N-1} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t+1,m}} \frac{\partial \hat{\alpha}_{t+1,m}}{\partial \hat{\alpha}_{t,j}}. \quad (54)$$

Substituting this into (53) and exchanging the order of summation, which is permissible because the sums are finite,

$$D_{t,j}^{(\alpha)} = \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t,j}} + \sum_{t'=t+1}^{T-1} \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t',k}} \sum_{m=0}^{N-1} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t+1,m}} \frac{\partial \hat{\alpha}_{t+1,m}}{\partial \hat{\alpha}_{t,j}} \quad (55)$$

$$= \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t,j}} + \sum_{m=0}^{N-1} \frac{\partial \hat{\alpha}_{t+1,m}}{\partial \hat{\alpha}_{t,j}} \underbrace{\left(\sum_{t'=t+1}^{T-1} \sum_{k=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t',k}} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t+1,m}} \right)}_{D_{t+1,m}^{(\alpha)} \text{ by (50)}} \quad (56)$$

$$= \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t,j}} + \sum_{m=0}^{N-1} \frac{\partial \hat{\alpha}_{t+1,m}}{\partial \hat{\alpha}_{t,j}} D_{t+1,m}^{(\alpha)}. \quad (57)$$

The bracketed factor is exactly the definition (50) evaluated at $(t+1, m)$: its outer sum starts at $t+1$ and its inner derivative is taken with respect to $\hat{\alpha}_{t+1,m}$. Recognizing it is what turns the double sum into a recursion.

At the last frame the outer sum of (50) contains only $t' = T-1$, so

$$D_{T-1,j}^{(\alpha)} = \frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{T-1,j}}, \quad (58)$$

which (57) also produces if one sets $D_{T,\cdot}^{(\alpha)} = 0$. The recursion therefore runs backwards in frame index, opposite to the forward pass that produced $\hat{\alpha}$.

3.3.4 Making the recursion explicit

Everything so far holds for any set of admissible predecessors. We now put $\mathcal{K}$ to work, first to reduce the sum in (57) to at most three terms and then to evaluate the factors that survive.

By (35) the factor $\partial\hat{\alpha}_{t+1,m}/\partial\hat{\alpha}_{t,j}$ is nonzero only when $j \in \mathcal{K}(m)$, that is, when $m - j \in \{0, 1, 2\}$ with $m = j + 2$ admitted only for $j + 1 \in \mathcal{S}$. Writing

$$w_{t,k \leftarrow j}^{(\alpha)} \equiv \frac{\partial\hat{\alpha}_{t,k}}{\partial\hat{\alpha}_{t-1,j}}, \quad t \geq 1, \quad (59)$$

and enumerating the cases in which each successor lies in range,

$$D_{t,j}^{(\alpha)} = \frac{\partial\mathcal{L}}{\partial\hat{\alpha}_{t,j}} + \begin{cases} w_{t+1,j \leftarrow j}^{(\alpha)} D_{t+1,j}^{(\alpha)} + w_{t+1,j+1 \leftarrow j}^{(\alpha)} D_{t+1,j+1}^{(\alpha)} + w_{t+1,j+2 \leftarrow j}^{(\alpha)} D_{t+1,j+2}^{(\alpha)}, & j \leq N-3, \quad j+1 \in \mathcal{S}, \\ w_{t+1,j \leftarrow j}^{(\alpha)} D_{t+1,j}^{(\alpha)} + w_{t+1,j+1 \leftarrow j}^{(\alpha)} D_{t+1,j+1}^{(\alpha)}, & j \leq N-3, \quad j+1 \notin \mathcal{S}, \\ w_{t+1,N-2 \leftarrow N-2}^{(\alpha)} D_{t+1,N-2}^{(\alpha)} + w_{t+1,N-1 \leftarrow N-2}^{(\alpha)} D_{t+1,N-1}^{(\alpha)}, & j = N-2, \\ w_{t+1,N-1 \leftarrow N-1}^{(\alpha)} D_{t+1,N-1}^{(\alpha)}, & j = N-1, \end{cases} \quad (60)$$

for $0 \leq t \leq T-2$. At $j = N-2$ a skip would land on token N , out of range, and at $j = N-1$ only a stay is available. The case structure mirrors that of the backward recursion (18), which is no coincidence: both enumerate the successors of a state rather than its predecessors.

It remains to evaluate the weights (59). By (30) applied to (35), in which the potential is additive and so contributes nothing, each weight is the softmax of the candidate list of its destination:

$$w_{t,k \leftarrow j}^{(\alpha)} = \frac{e^{\hat{\alpha}_{t-1,j}}}{\sum_{j' \in \mathcal{K}(k)} e^{\hat{\alpha}_{t-1,j'}}} \quad \text{for } j \in \mathcal{K}(k), \quad w_{t,k \leftarrow j}^{(\alpha)} = 0 \quad \text{otherwise.} \quad (61)$$

By (31) the weights of each destination sum to one,

$$\sum_{j \in \mathcal{K}(k)} w_{t,k \leftarrow j}^{(\alpha)} = 1, \quad (62)$$

so (60) distributes each $D_{t+1,m}^{(\alpha)}$ among the predecessors of m without changing its total.

Multiplying numerator and denominator by $e^{\hat{\phi}(t,k)}$ and using (35) for the denominator puts the weight in a form that displays its meaning:

$$w_{t,k \leftarrow j}^{(\alpha)} = \frac{e^{\hat{\alpha}_{t-1,j} + \hat{\phi}(t,k)}}{e^{\hat{\alpha}_{t,k}}} = e^{\hat{\alpha}_{t-1,j} + \hat{\phi}(t,k) - \hat{\alpha}_{t,k}}. \quad (63)$$

By (10) the numerator is the mass of the prefixes reaching (t,k) from j and the denominator that of all prefixes reaching (t,k) , so the weight is the conditional probability that a prefix ending at (t,k) came from j . The adjoint recursion propagates gradient backwards along the same conditional distribution the forward pass used to accumulate mass.

One case remains outside (50). That reduction assumed $t \geq 1$, since Lemma 6 read $\hat{\phi}(t,j)$ off (35). At $t = 0$ the initialization (8) governs instead, and there $\hat{\phi}(0,j)$ occurs only for the

admissible start states: for $j = 0$ always, and for $j = 1$ when $0 \in \mathcal{S}$. For those j the occurrence is again the whole of $\hat{\alpha}_{0,j}$, so $\partial \hat{\alpha}_{0,j} / \partial \hat{\phi}(0, j) = 1$ and the argument of Section 3.3.2 goes through unchanged; for every other j the value $\hat{\alpha}_{0,j} = -\infty$ carries no dependence on $\hat{\phi}(0, j)$, which by Lemma 6 occurs nowhere else, so the branch gradient vanishes. Hence

$$g_{0,j}^{(\alpha)} = \begin{cases} D_{0,0}^{(\alpha)}, & j = 0, \\ D_{0,1}^{(\alpha)}, & j = 1 \text{ and } 0 \in \mathcal{S}, \\ 0, & \text{otherwise,} \end{cases} \quad (64)$$

while $g_{t,j}^{(\alpha)} = D_{t,j}^{(\alpha)}$ for all $t \geq 1$. The alpha branch is thus computed by running (58) and (60) backwards over frames and reading off $g^{(\alpha)}$ with the exception (64) at the first frame.

3.4 Beta branch

This section evaluates the second sum of (29),

$$g_{t,k}^{(\beta)} = \sum_{t'=0}^{T-1} \sum_{j=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t',j}} \frac{\partial \hat{\beta}_{t',j}}{\partial \hat{\phi}(t, k)}. \quad (65)$$

The plan is the same as in Section 3.3, and the first two structural facts carry over by symmetry. The last two do not, and the difference changes the shape of the result: where the alpha branch produced a single adjoint, the beta branch produces a pair of quantities that must be advanced together.

3.4.1 Dependency structure of the backward recursion

Name the candidate set that (18) passes to LSE. Collecting its four cases, the *admissible successors* of a token j are

$$\mathcal{M}(j) = \begin{cases} \{j, j+1, j+2\}, & j \leq N-3, \ j+1 \in \mathcal{S}, \\ \{j, j+1\}, & j \leq N-3, \ j+1 \notin \mathcal{S}, \\ \{N-2, N-1\}, & j = N-2, \\ \{N-1\}, & j = N-1, \end{cases} \quad (66)$$

so that the recursion reads

$$\hat{\beta}_{s,j} = \text{LSE}_{k \in \mathcal{M}(j)} (\hat{\phi}(s+1, k) + \hat{\beta}_{s+1,k}), \quad s \leq T-2. \quad (67)$$

Note that $\mathcal{M}$ and $\mathcal{K}$ describe the same relation from opposite sides: comparing (66) with (34),

$$k \in \mathcal{M}(j) \iff j \in \mathcal{K}(k), \quad (68)$$

both saying that the transition $j \rightarrow k$ is admissible.

Two properties mirror Lemmas 3 and 4, with the direction of time reversed.

Lemma 7. $\hat{\beta}_{t',j}$ is a function of $\hat{\phi}(s, \cdot)$ for $s > t'$ only. Consequently

$$\frac{\partial \hat{\beta}_{t',j}}{\partial \hat{\phi}(t, k)} = 0 \quad \text{whenever } t \leq t'. \quad (69)$$

Proof. Reverse induction on t' , symmetric to Lemma 3. At $t' = T - 1$ the initialization (14) is a constant and involves no potential. For $t' \leq T - 2$ the recursion (67) forms $\hat{\beta}_{t',j}$ from $\hat{\phi}(t' + 1, \cdot)$ and from $\hat{\beta}_{t'+1,\cdot}$, the latter involving only $\hat{\phi}(s, \cdot)$ with $s > t' + 1$ by hypothesis. $\square$

The frame maps run the other way too. Define $S_s : \mathbb{R}^N \rightarrow \mathbb{R}^N$ by

$$(S_s(v))_j = \text{LSE}_{k \in \mathcal{M}(j)}(\hat{\phi}(s + 1, k) + v_k), \quad (70)$$

whose constants are the potentials $\hat{\phi}(s + 1, \cdot)$, so that (67) becomes $\hat{\beta}_{s,\cdot} = S_s(\hat{\beta}_{s+1,\cdot})$. Applying this repeatedly from t' upwards gives, for $t' \leq \tau$,

$$\hat{\beta}_{t',\cdot} = (S_{t'} \circ S_{t'+1} \circ \cdots \circ S_{\tau-1})(\hat{\beta}_{\tau,\cdot}) \equiv \tilde{C}_{\tau \rightarrow t'}(\hat{\beta}_{\tau,\cdot}), \quad (71)$$

the identity when $\tau = t'$, and with constants $\hat{\phi}(s, \cdot)$ for $t' < s \leq \tau$.

Lemma 8. For $t' \leq \tau \leq t$,

$$\frac{\partial \hat{\beta}_{t',j}}{\partial \hat{\beta}_{t,k}} = \sum_{m=0}^{N-1} \frac{\partial \hat{\beta}_{t',j}}{\partial \hat{\beta}_{\tau,m}} \frac{\partial \hat{\beta}_{\tau,m}}{\partial \hat{\beta}_{t,k}}. \quad (72)$$

Proof. Symmetric to Lemma 4, using (71) in place of (41). $\square$

The next two properties are where the branch parts company with Section 3.3. In (35) the potential sits outside the LSE, added to the one forward variable that carries its indices. In (67) it sits inside, paired with a backward variable of the *next* frame, and appears in the candidate list of every state that can reach it.

Lemma 9. For $t \geq 1$ and $t' \leq t - 1$,

$$\frac{\partial \hat{\beta}_{t',j}}{\partial \hat{\phi}(t,k)} = \sum_{m=0}^{N-1} \frac{\partial \hat{\beta}_{t',j}}{\partial \hat{\beta}_{t-1,m}} \frac{\partial \hat{\beta}_{t-1,m}}{\partial \hat{\phi}(t,k)}. \quad (73)$$

Proof. Take $\tau = t - 1$ in (71). The constants of $\tilde{C}_{t-1 \rightarrow t'}$ are the potentials $\hat{\phi}(s, \cdot)$ with $t' < s \leq t - 1$, none of which is $\hat{\phi}(t, \cdot)$, so $\hat{\phi}(t, k)$ can act on $\hat{\beta}_{t',j}$ only through the components $\hat{\beta}_{t-1,m}$, and Lemma 13 applies as before. $\square$

The routing frame is $t - 1$, not t : by the convention (13) a backward variable excludes the potential of its own state, so $\hat{\phi}(t, \cdot)$ first appears one frame earlier than its own index. What happens at that frame is the second difference.

Lemma 10. For $0 \leq s \leq T - 2$, write

$$w_{s,j \rightarrow k}^{(\beta)} \equiv \frac{\partial \hat{\beta}_{s,j}}{\partial \hat{\beta}_{s+1,k}}. \quad (74)$$

Then

$$\frac{\partial \hat{\beta}_{s,j}}{\partial \hat{\phi}(s+1,k)} = w_{s,j \rightarrow k}^{(\beta)}, \quad (75)$$

and this quantity is nonzero only for $k \in \mathcal{M}(j)$, equivalently only for $j \in \mathcal{K}(k)$.

Proof. In (67) the two variables $\hat{\phi}(s+1, k)$ and $\hat{\beta}_{s+1, k}$ occur only in the single candidate $\hat{\phi}(s+1, k) + \hat{\beta}_{s+1, k}$, and only when $k \in \mathcal{M}(j)$. Differentiating that candidate with respect to either variable gives 1, so by the chain rule both derivatives of $\hat{\beta}_{s, j}$ equal the derivative of LSE with respect to that candidate, namely its softmax weight in the list $\{\hat{\phi}(s+1, k') + \hat{\beta}_{s+1, k'} : k' \in \mathcal{M}(j)\}$. The two are therefore equal, and vanish when $k \notin \mathcal{M}(j)$; the restatement in terms of $\mathcal{K}$ is (68). $\square$

Lemma 10 replaces Lemma 6, and differs from it in both respects. There is no Kronecker delta: a fixed k makes (75) nonzero for every $j \in \mathcal{K}(k)$, of which there may be three. And the derivative with respect to the potential coincides with the derivative with respect to a backward variable, because the two enter only as their sum. The first fact will keep a sum from collapsing; the second will tie the two quantities of the branch together.

As in Section 3.3.1, the sums in (72) and (73) run over all N tokens, since they rest only on the layered structure (71). It is Section 3.4.4 that puts $\mathcal{M}$ to work.

3.4.2 Reduction to a coupled pair

Fix t and k . Starting from (65),

$$g_{t, k}^{(\beta)} = \sum_{t'=0}^{T-1} \sum_{j=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t', j}} \frac{\partial \hat{\beta}_{t', j}}{\partial \hat{\phi}(t, k)} \quad (76)$$

$$= \sum_{t'=0}^{t-1} \sum_{j=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t', j}} \frac{\partial \hat{\beta}_{t', j}}{\partial \hat{\phi}(t, k)} \quad \text{by Lemma 7} \quad (77)$$

$$= \sum_{t'=0}^{t-1} \sum_{j=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t', j}} \sum_{m=0}^{N-1} \frac{\partial \hat{\beta}_{t', j}}{\partial \hat{\beta}_{t-1, m}} \frac{\partial \hat{\beta}_{t-1, m}}{\partial \hat{\phi}(t, k)} \quad \text{by Lemma 9} \quad (78)$$

$$= \sum_{m=0}^{N-1} \frac{\partial \hat{\beta}_{t-1, m}}{\partial \hat{\phi}(t, k)} \underbrace{\left(\sum_{t'=0}^{t-1} \sum_{j=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t', j}} \frac{\partial \hat{\beta}_{t', j}}{\partial \hat{\beta}_{t-1, m}} \right)}_{\equiv D_{t-1, m}^{(\beta)}} \quad (79)$$

$$= \sum_{m \in \mathcal{K}(k)} w_{t-1, m \rightarrow k}^{(\beta)} D_{t-1, m}^{(\beta)}. \quad \text{by Lemma 10} \quad (80)$$

Line (77) drops the terms in which the potential lies in the past of the backward variable. Line (78) routes the remainder through frame $t-1$, and (79) exchanges the order of summation and names the resulting bracket, defining the *beta adjoint*

$$D_{s, m}^{(\beta)} \equiv \sum_{t'=0}^s \sum_{j=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t', j}} \frac{\partial \hat{\beta}_{t', j}}{\partial \hat{\beta}_{s, m}}. \quad (81)$$

Line (80) then substitutes (75) and discards the terms in which the transition is inadmissible.

Comparing (80) with its counterpart (50) shows the structural difference. There the sum collapsed to one term and the branch gradient was the adjoint itself; here no delta appears,

and the branch gradient is a weighted sum of up to three adjoints of the previous frame. Two quantities are therefore in play rather than one.

At the first frame the outer sum of (65) is empty after (77), so

$$g_{0,k}^{(\beta)} = 0, \quad (82)$$

which is (69) read at $t = 0$: the backward recursion never consumes the potentials of the first frame.

3.4.3 The coupled recursion

It remains to compute $D^{(\beta)}$. The derivation follows Section 3.3.3, and the second half of Lemma 10 then identifies the result with something already named.

Split the outer sum of (81) at its last term. For $t' = t$ the inner derivative is $\partial \hat{\beta}_{t,j} / \partial \hat{\beta}_{t,m} = \delta_{jm}$, since by (67) the components of one frame are each computed from the following frame and not from one another, so that term collapses to $\partial \mathcal{L} / \partial \hat{\beta}_{t,m}$. Every remaining term has $t' \leq t-1$, so Lemma 8 applies with $\tau = t-1$, and exchanging the order of summation as in (79),

$$D_{t,m}^{(\beta)} = \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t,m}} + \sum_{n=0}^{N-1} \frac{\partial \hat{\beta}_{t-1,n}}{\partial \hat{\beta}_{t,m}} \underbrace{\left(\sum_{t'=0}^{t-1} \sum_{j=0}^{N-1} \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t',j}} \frac{\partial \hat{\beta}_{t',j}}{\partial \hat{\beta}_{t-1,n}} \right)}_{D_{t-1,n}^{(\beta)} \text{ by (81)}} \quad (83)$$

$$= \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t,m}} + \sum_{n \in \mathcal{K}(m)} w_{t-1,n \rightarrow m}^{(\beta)} D_{t-1,n}^{(\beta)}, \quad (84)$$

the second line using (74) and discarding inadmissible transitions.

Proposition 11. *For every t and m ,*

$$D_{t,m}^{(\beta)} = \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t,m}} + g_{t,m}^{(\beta)}. \quad (85)$$

Proof. The sum in (84) and the sum in (80) are the same expression: both run over $n \in \mathcal{K}(m)$, both weight $D_{t-1,n}^{(\beta)}$, and both use the factor $w_{t-1,n \rightarrow m}^{(\beta)}$. That the two arrive at the same factor from different derivatives — one with respect to $\hat{\beta}_{t,m}$ and one with respect to $\hat{\phi}(t, m)$ — is exactly (75). Substituting (80) into (84) gives the claim. $\square$

Equations (80) and (85) form a single recursion in the two unknowns, advancing forwards in frame index:

$$g_{t,m}^{(\beta)} = \sum_{n \in \mathcal{K}(m)} w_{t-1,n \rightarrow m}^{(\beta)} D_{t-1,n}^{(\beta)}, \quad D_{t,m}^{(\beta)} = \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t,m}} + g_{t,m}^{(\beta)}, \quad (86)$$

started from (82) and hence

$$D_{0,m}^{(\beta)} = \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{0,m}}. \quad (87)$$

Each frame is produced from the previous one by a gather over $\mathcal{K}$ followed by the addition of one entry of the upstream array. The direction is opposite to that of the alpha branch, and again opposite to the pass that produced the variable being differentiated.

3.4.4 Making the recursion explicit

Everything above holds for any admissible transition structure. Substituting (34) into (86) and enumerating the cases in which each predecessor lies in range gives, for $1 \leq t \leq T - 1$,

$$g_{t,k}^{(\beta)} = \begin{cases} w_{t-1,k \rightarrow k}^{(\beta)} D_{t-1,k}^{(\beta)} + w_{t-1,k-1 \rightarrow k}^{(\beta)} D_{t-1,k-1}^{(\beta)} \\ \quad + w_{t-1,k-2 \rightarrow k}^{(\beta)} D_{t-1,k-2}^{(\beta)}, & k \geq 2, \quad k-1 \in \mathcal{S}, \\ w_{t-1,k \rightarrow k}^{(\beta)} D_{t-1,k}^{(\beta)} + w_{t-1,k-1 \rightarrow k}^{(\beta)} D_{t-1,k-1}^{(\beta)}, & k \geq 2, \quad k-1 \notin \mathcal{S}, \\ w_{t-1,1 \rightarrow 1}^{(\beta)} D_{t-1,1}^{(\beta)} + w_{t-1,0 \rightarrow 1}^{(\beta)} D_{t-1,0}^{(\beta)}, & k = 1, \\ w_{t-1,0 \rightarrow 0}^{(\beta)} D_{t-1,0}^{(\beta)}, & k = 0. \end{cases} \quad (88)$$

The case structure is that of the forward recursion (11), since both enumerate the predecessors of a state; this is the mirror of the observation made after (60).

The weights follow from (30) applied to (67), whose candidates are the sums $\hat{\phi}(s+1, k) + \hat{\beta}_{s+1,k}$:

$$w_{s,j \rightarrow k}^{(\beta)} = \frac{e^{\hat{\phi}(s+1,k) + \hat{\beta}_{s+1,k}}}{\sum_{k' \in \mathcal{M}(j)} e^{\hat{\phi}(s+1,k') + \hat{\beta}_{s+1,k'}}} \quad \text{for } k \in \mathcal{M}(j), \quad 0 \quad \text{otherwise}, \quad (89)$$

and by (31) the weights of each source sum to one,

$$\sum_{k \in \mathcal{M}(j)} w_{s,j \rightarrow k}^{(\beta)} = 1. \quad (90)$$

Using (67) for the denominator puts the weight in the form that displays its meaning,

$$w_{s,j \rightarrow k}^{(\beta)} = \frac{e^{\hat{\phi}(s+1,k) + \hat{\beta}_{s+1,k}}}{e^{\hat{\beta}_{s,j}}} = e^{\hat{\phi}(s+1,k) + \hat{\beta}_{s+1,k} - \hat{\beta}_{s,j}}, \quad (91)$$

whose numerator is, by (17), the mass of the suffixes leaving (s, j) through k and whose denominator is that of all suffixes leaving (s, j) . The weight is therefore the conditional probability that a suffix starting at (s, j) moves to k , and (86) propagates gradient forwards along the same distribution the backward pass used to accumulate mass.

Unlike the alpha branch, no frame needs separate treatment. The exception there arose because the initialization (8) defined $\hat{\alpha}_0$ directly in terms of potentials, so (44) had to be checked against a different formula. Here (14) involves no potential at all, and the only boundary condition is (82), which the recursion already incorporates.

3.4.5 Contrast with the alpha branch

The two branches differ in three linked ways, all traceable to where the potential sits. In (35) it is outside the LSE and carries the indices of the forward variable it is added to, so Lemma 6 yields a Kronecker delta, the sum in (49) collapses, and one quantity suffices. In (67) it is inside, paired with a backward variable of the next frame and shared by every state that can reach it, so Lemma 10 yields no delta and $g^{(\beta)}$ and $D^{(\beta)}$ must advance together. The same lemma supplies the compensation: because $\hat{\phi}(s+1, k)$ and $\hat{\beta}_{s+1,k}$ enter only as their sum, the gather that produces $g^{(\beta)}$ is the gather that advances $D^{(\beta)}$, and the coupling costs one addition per state. Finally the recursions run in opposite directions, each the reverse of the pass it differentiates.

3.5 Closing the loop

The two branches were derived for an arbitrary downstream computation $\mathcal{L}$. Specializing to the simplest one, $L = -\log Z$, turns the machinery of Sections 3.3 and 3.4 into a statement about the posterior marginals of Section 2.4. This is not an objective the branches are needed for — were it the only one, the marginals would already be its gradient — but it is the case in which both can be evaluated in closed form, and it therefore serves as a check on an implementation of them.

3.5.1 The upstream gradients of the log partition function

Take

$$L = -\log Z = -\hat{\alpha}_{T-1, N-1} \quad (92)$$

by (12). As a function of the two arrays, $\mathcal{L}$ then reads a single entry of the first and ignores the second, so

$$\frac{\partial \mathcal{L}}{\partial \hat{\alpha}_{t,j}} = -\delta_{t, T-1} \delta_{j, N-1}, \quad \frac{\partial \mathcal{L}}{\partial \hat{\beta}_{t,j}} = 0. \quad (93)$$

The beta branch vanishes immediately. With zero upstream, (87) gives $D_{0,\cdot}^{(\beta)} = 0$, and (86) then propagates zeros forwards, so

$$g_{t,j}^{(\beta)} = 0 \quad \text{for all } t, j. \quad (94)$$

This is as expected: $\log Z$ was read off $\hat{\alpha}$, and by (26) the backward variables took no part in producing it.

3.5.2 The alpha branch returns the marginals

Proposition 12. Under (92),

$$D_{t,j}^{(\alpha)} = -\gamma_{t,j} \quad \text{for all } t, j, \quad (95)$$

and consequently

$$\frac{\partial \log Z}{\partial \hat{\phi}(t, j)} = \gamma_{t,j}. \quad (96)$$

Proof sketch. Reverse induction on t . At $t = T - 1$ the initializations (58) and (14) both single out $j = N - 1$. In the inductive step the upstream term of (57) vanishes, and substituting (63) into $w_{t+1, m \leftarrow j}^{(\alpha)} \gamma_{t+1, m}$ cancels the forward variable of frame $t + 1$; summing the remainder over $m \in \mathcal{M}(j)$ reassembles $e^{\hat{\beta}_{t,j}}$ through (67), which is $\gamma_{t,j}$. Appendix D gives the details. $\square$

3.5.3 Consequences

Equation (96) is the familiar statement that the gradient of a log partition function with respect to a node potential is the corresponding marginal, and it closes the development: the forward recursion computes $\log Z$, the backward recursion completes it into γ , and differentiating the first with respect to its inputs returns the second.

It also gives a check that needs no finite-difference reference. Running the two branches with the upstream gradients (93) and comparing the output against γ formed from the two forward passes exercises the recursions, the transition weights and the boundary cases at once.

Appendix

A The chain rule for a shared argument

Every differentiation below is an instance of one rule, stated here in the form in which it is applied.

Lemma 13. *Let $u_0, \dots, u_{n-1}$ be real-valued functions of a variable x , differentiable at x , and let f be a real-valued function of n arguments, differentiable at $(u_0(x), \dots, u_{n-1}(x))$. Then the composition $F(x) = f(u_0(x), \dots, u_{n-1}(x))$ satisfies*

$$\frac{dF}{dx} = \sum_{m=0}^{n-1} \frac{\partial f}{\partial v_m} \bigg|_{v=u(x)} \frac{du_m}{dx}, \quad (97)$$

where $\partial f / \partial v_m$ denotes the partial derivative of f in its m -th argument, the remaining arguments held fixed.

Proof. Perturbing x by Δx changes each argument by $\Delta u_m = (du_m/dx) \Delta x + o(\Delta x)$. Differentiability of f at $u(x)$ means that its increment is the sum of its first-order responses to the individual arguments,

$$\Delta F = \sum_{m=0}^{n-1} \frac{\partial f}{\partial v_m} \Delta u_m + o(\|\Delta u\|). \quad (98)$$

Substituting, dividing by Δx and letting $\Delta x \rightarrow 0$ gives (97). $\square$

Two features are worth isolating, since both recur. First, x may reach f along several routes at once, one per argument, and the routes contribute additively rather than interfering; the case $n = 2$ is the familiar $F(x) = f(g(x), h(x))$ with $dF/dx = (\partial f / \partial a) g'(x) + (\partial f / \partial b) h'(x)$. Second, the sum runs over *every* argument of f , not only over those on which f actually depends: an argument that f ignores contributes a vanishing partial derivative rather than being excluded in advance. That is why the sums in Sections 3.3 and 3.4 range over all N tokens even where at most three of the terms survive.

B Proofs for the posterior marginals

This appendix proves Propositions 1 and 2 of Section 2.4, restated here for reference.

Proposition 1. *For every t and j , $\gamma_{t,j} = \exp(\hat{\alpha}_{t,j} + \hat{\beta}_{t,j} - \log Z)$.*

Proof. Fix t and j .

Step 1: cutting a path at frame t is a bijection. Map a path $z \in \mathcal{P}_M^{t,j}$ to the pair consisting of its prefix and suffix at frame t , which overlap in the single state $z_t = j$:

$$\pi(z) = ((z_0, \dots, z_t), (z_t, \dots, z_{T-1})). \quad (99)$$

We claim π is a bijection from $\mathcal{P}_M^{t,j}$ onto $\mathcal{A}_{t,j} \times \mathcal{B}_{t,j}$.

Well defined. The first component satisfies (1) for $0 \leq s \leq t-1$ and satisfies (2), both inherited from z , and ends at $z_t = j$; hence it lies in $\mathcal{A}_{t,j}$. The second satisfies (1) for $t \leq s \leq T-2$, starts at j , and ends at $z_{T-1} = N-1$; hence it lies in $\mathcal{B}_{t,j}$.

Injective. From $\pi(z)$ the path z is recovered by concatenating the two components along their shared state, so $\pi(z) = \pi(z')$ forces $z = z'$.

Surjective. Let $a = (a_0, \dots, a_t) \in \mathcal{A}_{t,j}$ and $b = (b_t, \dots, b_{T-1}) \in \mathcal{B}_{t,j}$ be arbitrary. Then $a_t = b_t = j$, so

$$z = (a_0, \dots, a_{t-1}, j, b_{t+1}, \dots, b_{T-1}) \quad (100)$$

is well defined, and $\pi(z) = (a, b)$ by construction. It remains to check $z \in \mathcal{P}_M^{t,j}$. The start condition (2) holds because $z_0 = a_0$ and $a \in \mathcal{A}_{t,j}$. The terminal condition holds because $z_{T-1} = b_{T-1} = N-1$. For (1), consider each s in turn: for $s \leq t-2$ the pair $(z_s, z_{s+1}) = (a_s, a_{s+1})$ is admissible because a is; for $s = t-1$ the pair is $(a_{t-1}, j) = (a_{t-1}, a_t)$, again admissible because a is; for $s = t$ the pair is $(j, b_{t+1}) = (b_t, b_{t+1})$, admissible because b is; and for $s \geq t+1$ the pair is (b_s, b_{s+1}) , admissible for the same reason. No constraint couples a_{t-1} to b_{t+1} , since (1) relates only consecutive states. Hence $z \in \mathcal{P}_M^{t,j}$.

Step 2: the log-weight splits with no term counted twice. Let $z \in \mathcal{P}_M^{t,j}$ with $\pi(z) = (a, b)$. Splitting the index range of (4) at t ,

$$\sum_{s=0}^{T-1} \hat{\phi}(s, z_s) = \underbrace{\sum_{s=0}^t \hat{\phi}(s, a_s)}_{\text{prefix; includes } \hat{\phi}(t,j)} + \underbrace{\sum_{s=t+1}^{T-1} \hat{\phi}(s, b_s)}_{\text{suffix; excludes } \hat{\phi}(t,j)}, \quad (101)$$

which uses $z_s = a_s$ for $s \leq t$ and $z_s = b_s$ for $s \geq t+1$. The ranges $\{0, \dots, t\}$ and $\{t+1, \dots, T-1\}$ are disjoint and cover $\{0, \dots, T-1\}$, so every term of the left-hand side appears exactly once on the right. In particular $\hat{\phi}(t, j)$ appears once, on the prefix side, which is where the exclusion built into (13) leaves it.

Step 3: sum over the product set. Reindexing the sum in (23) by Step 1 and splitting each log-weight by (101),

$$\sum_{z \in \mathcal{P}_M^{t,j}} \Psi(z) = \sum_{z \in \mathcal{P}_M^{t,j}} \exp \left(\sum_{s=0}^{T-1} \hat{\phi}(s, z_s) \right) \quad (102)$$

$$= \sum_{(a,b) \in \mathcal{A}_{t,j} \times \mathcal{B}_{t,j}} \exp \left(\sum_{s=0}^t \hat{\phi}(s, a_s) + \sum_{s=t+1}^{T-1} \hat{\phi}(s, b_s) \right) \quad (103)$$

$$= \sum_{a \in \mathcal{A}_{t,j}} \sum_{b \in \mathcal{B}_{t,j}} \exp \left(\sum_{s=0}^t \hat{\phi}(s, a_s) \right) \exp \left(\sum_{s=t+1}^{T-1} \hat{\phi}(s, b_s) \right) \quad (104)$$

$$= \left(\sum_{a \in \mathcal{A}_{t,j}} \exp \sum_{s=0}^t \hat{\phi}(s, a_s) \right) \left(\sum_{b \in \mathcal{B}_{t,j}} \exp \sum_{s=t+1}^{T-1} \hat{\phi}(s, b_s) \right) \quad (105)$$

$$= e^{\hat{\alpha}_{t,j}} e^{\hat{\beta}_{t,j}}. \quad (106)$$

Line (105) holds because the first factor depends only on a and the second only on b , so each may be pulled out of the sum over the other index. The last line applies the definitions (7) and (13).

Conclusion. Dividing (106) by Z and comparing with (23),

$$\gamma_{t,j} = \frac{e^{\hat{\alpha}_{t,j}} e^{\hat{\beta}_{t,j}}}{Z} = \exp(\hat{\alpha}_{t,j} + \hat{\beta}_{t,j} - \log Z). \quad (107)$$

□

Proposition 2. For every t , $\sum_{j=0}^{N-1} \gamma_{t,j} = 1$.

Proof. Fix t . Every path assigns frame t to exactly one token, so the sets $\mathcal{P}_M^{t,0}, \dots, \mathcal{P}_M^{t,N-1}$ defined in (23) are pairwise disjoint, and every $z \in \mathcal{P}_M$ belongs to $\mathcal{P}_M^{t,z_t}$. Hence

$$\mathcal{P}_M = \bigsqcup_{j=0}^{N-1} \mathcal{P}_M^{t,j}. \quad (108)$$

Summing the identity (106) over j and using (108) to merge the inner sums,

$$\sum_{j=0}^{N-1} e^{\hat{\alpha}_{t,j} + \hat{\beta}_{t,j}} \stackrel{(106)}{=} \sum_{j=0}^{N-1} \sum_{z \in \mathcal{P}_M^{t,j}} \Psi(z) \stackrel{(108)}{=} \sum_{z \in \mathcal{P}_M} \Psi(z) \stackrel{(5)}{=} Z, \quad (109)$$

the middle step being valid precisely because the inner sums range over disjoint sets whose union is $\mathcal{P}_M$, so each path is counted once. Dividing (109) by Z and applying Proposition 1 term by term,

$$\sum_{j=0}^{N-1} \gamma_{t,j} = \sum_{j=0}^{N-1} \frac{e^{\hat{\alpha}_{t,j} + \hat{\beta}_{t,j}}}{Z} = \frac{1}{Z} \sum_{j=0}^{N-1} e^{\hat{\alpha}_{t,j} + \hat{\beta}_{t,j}} = \frac{Z}{Z} = 1. \quad (110)$$

□

C What the alpha recursion computes

The adjoint of Section 3.3 is, like $\hat{\alpha}$ and $\hat{\beta}$ themselves, a sum over paths, and the recursion that evaluates it is the factorization that collapses that sum. Nothing below is needed to compute a gradient; it is recorded because it explains the cost.

Unrolling (57) shows that the adjoint, like $\hat{\alpha}$ and $\hat{\beta}$ themselves, is a sum over paths. Call $(m_t, m_{t+1}, \dots, m_{t'})$ an *admissible segment* from (t, j) to (t', k) when $m_t = j$, $m_{t'} = k$, and consecutive states satisfy (1).

Proposition 14. For $t \leq t'$,

$$\frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t,j}} = \sum_{\substack{\text{admissible segments} \\ (m_t, \dots, m_{t'}) \text{ from } (t,j) \text{ to } (t',k)}} \prod_{s=t+1}^{t'} w_{s, m_s \leftarrow m_{s-1}}^{(\alpha)}. \quad (111)$$

Proof. Induction on $d = t' - t$. For $d = 0$ the left side is δ_{kj} . On the right, if $k = j$ the only segment is the single state (j) , whose product is empty and therefore equal to 1; if $k \neq j$ there is no segment and the sum is empty, hence 0. The two agree.

Assume (111) holds for d and let $t' - t = d + 1$. By Lemma 4 with $\tau = t + 1$,

$$\frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t,j}} = \sum_{m=0}^{N-1} \frac{\partial \hat{\alpha}_{t',k}}{\partial \hat{\alpha}_{t+1,m}} w_{t+1,m \leftarrow j}^{(\alpha)}, \quad (112)$$

using (59) for the second factor. The first factor spans $t' - (t + 1) = d$ frames, so the inductive hypothesis expands it into a sum over admissible segments from $(t + 1, m)$ to (t', k) . Prefixing the state j to such a segment yields an admissible segment from (t, j) to (t', k) precisely when $w_{t+1,m \leftarrow j}^{(\alpha)} \neq 0$, that is, when j is an admissible predecessor of m ; conversely every admissible segment from (t, j) to (t', k) arises this way from exactly one m , namely its state at frame $t + 1$. The correspondence is bijective and multiplies the product by the factor $w_{t+1,m \leftarrow j}^{(\alpha)}$, which is the term supplied above. Summing over m therefore reproduces (111) at $d + 1$. $\square$

Substituting (111) into (50) expresses $D_{t,j}^{(\alpha)}$ as an upstream-weighted sum over all admissible segments leaving (t, j) . The number of such segments grows combinatorially in $T - t$, which is why (50) is not evaluated directly, and the recursion (60) is precisely the factorization that collapses that sum: at each frame it retains one number per state instead of one number per segment, exactly as (11) does for the forward pass. The cost is $O(TN)$, with the constant set by the at most three transitions per state.

D Proof of the log partition gradient

This appendix proves Proposition 12 of Section 3.5, restated here for reference.

Proposition 12. *Under (92), $D_{t,j}^{(\alpha)} = -\gamma_{t,j}$ for all t and j , and consequently $\partial \log Z / \partial \hat{\phi}(t, j) = \gamma_{t,j}$.*

Proof. We prove (95) by reverse induction on t .

Base case $t = T - 1$. By (58) and (93), $D_{T-1,j}^{(\alpha)} = -\delta_{j,N-1}$. On the other side, Proposition 1 with the initialization (14) gives

$$\gamma_{T-1,j} = e^{\hat{\alpha}_{T-1,j} + \hat{\beta}_{T-1,j} - \log Z} = \begin{cases} e^{\hat{\alpha}_{T-1,N-1} - \log Z} = 1, & j = N - 1, \\ 0, & j \neq N - 1, \end{cases} \quad (113)$$

using $\hat{\beta}_{T-1,N-1} = 0$ and $\log Z = \hat{\alpha}_{T-1,N-1}$. The two agree.

Inductive step. Suppose $D_{t+1,m}^{(\alpha)} = -\gamma_{t+1,m}$ for all m . Since (93) vanishes at every frame before the last, the recursion (57) reduces to its second term, and restricting the sum to admissible transitions as in (60),

$$D_{t,j}^{(\alpha)} = \sum_{m: j \in \mathcal{K}(m)} w_{t+1,m \leftarrow j}^{(\alpha)} D_{t+1,m}^{(\alpha)} = - \sum_{m \in \mathcal{M}(j)} w_{t+1,m \leftarrow j}^{(\alpha)} \gamma_{t+1,m}, \quad (114)$$

the index set being rewritten by (68). Substituting (63) for the weight and Proposition 1 for the marginal, the summand is

$$w_{t+1,m \leftarrow j}^{(\alpha)} \gamma_{t+1,m} = e^{\hat{\alpha}_{t,j} + \hat{\phi}(t+1,m) - \hat{\alpha}_{t+1,m}} \cdot e^{\hat{\alpha}_{t+1,m} + \hat{\beta}_{t+1,m} - \log Z} = e^{\hat{\alpha}_{t,j} + \hat{\phi}(t+1,m) + \hat{\beta}_{t+1,m} - \log Z}, \quad (115)$$

in which the forward variable of frame $t + 1$ has cancelled. Summing over $m \in \mathcal{M}(j)$ and pulling out the factors that do not depend on m ,

$$\sum_{m \in \mathcal{M}(j)} w_{t+1, m \leftarrow j}^{(\alpha)} \gamma_{t+1, m} = e^{\hat{\alpha}_{t, j} - \log Z} \sum_{m \in \mathcal{M}(j)} e^{\hat{\phi}(t+1, m) + \hat{\beta}_{t+1, m}} \quad (116)$$

$$= e^{\hat{\alpha}_{t, j} - \log Z} \cdot e^{\hat{\beta}_{t, j}} \quad (117)$$

$$= \gamma_{t, j}, \quad (118)$$

the middle step being (67) exponentiated and the last Proposition 1. Hence $D_{t, j}^{(\alpha)} = -\gamma_{t, j}$, which completes the induction.

For (96), combine the two branches through (29). At $t \geq 1$ the alpha branch gives $g_{t, j}^{(\alpha)} = D_{t, j}^{(\alpha)}$ by (50), and the beta branch contributes nothing by (94), so $\partial L / \partial \hat{\phi}(t, j) = -\gamma_{t, j}$. At $t = 0$ the exception (64) sets $g_{0, j}^{(\alpha)} = 0$ for every j that is not an admissible start state; but those states have $\hat{\alpha}_{0, j} = -\infty$ by (8) and hence $\gamma_{0, j} = 0$, so the identity holds there as well. Negating gives (96). $\square$

E Numerically stable log-sum-exp and its derivative

The definitions of LSE and of its derivative (30) are stated in a form that overflows if used literally, since e^{x_i} is evaluated for values of x_i that may be large in magnitude. Both admit a shifted form that does not [4].

Writing $M = \max_i x_i$,

$$\text{LSE}(x_1, \dots, x_m) = M + \log \sum_{r=1}^m e^{x_r - M}, \quad \text{softmax}_i(x_1, \dots, x_m) = \frac{e^{x_i - M}}{\sum_{r=1}^m e^{x_r - M}}. \quad (119)$$

Every exponent is now at most zero, and the term r attaining the maximum contributes exactly one, so the sum lies in $[1, m]$. For the two-candidate case that dominates the recursions, (119) reduces to

$$\text{LSE}(a, b) = \max(a, b) + \log(1 + e^{-|a-b|}), \quad (120)$$

whose second term is evaluated with a $\log(1 + x)$ routine so that it stays accurate when $|a - b|$ is large and the correction is small.

Two considerations decide which of the equivalent expressions for the transition weights is evaluated. Both (61) and (63) are exact, the first a softmax over a candidate list and the second a single exponential of stored recursion outputs, and likewise (89) and (91). The implementation uses the softmax forms.

The first consideration is cancellation. A forward variable accumulates one potential per frame, so $\hat{\alpha}_{t, k}$ grows in magnitude roughly linearly in t , whereas the exponent of (63) is at most zero and typically of order one. Computing a small quantity as the difference of large ones costs relative precision [7], since the stored values carry an absolute error proportional to their magnitude and that error passes undiminished into the exponent. The shift in (119) instead subtracts a maximum taken within one candidate list, whose entries are forward variables of a single frame and hence of comparable size, removing the large common part rather than cancelling it.

The second is the treatment of unreachable states, which are represented by a large negative sentinel rather than by $-\infty$. Arithmetic on sentinels is not meaningful: if $\hat{\alpha}_{t-1,j}$ and $\hat{\alpha}_{t,k}$ are both the sentinel, their difference in (63) cancels and the formula returns $e^{\hat{\phi}(t,k)}$, a nonzero weight for a transition that does not exist. The softmax form admits a direct guard instead, returning zero for any candidate at or below the sentinel and zero for the whole list when none exceeds it.

References

- [1] Rohan Badlani, Adrian Lańcucki, Kevin J. Shih, Rafael Valle, Wei Ping, and Bryan Catanzaro. One TTS alignment to rule them all. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6092–6096, 2022.
- [2] Leonard E. Baum, Ted Petrie, George Soules, and Norman Weiss. A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains. *The Annals of Mathematical Statistics*, 41(1):164–171, 1970.
- [3] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.
- [4] Pierre Blanchard, Desmond J. Higham, and Nicholas J. Higham. Accurately computing the log-sum-exp and softmax functions. *IMA Journal of Numerical Analysis*, 41(4):2311–2330, 2021.
- [5] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 369–376, 2006.
- [6] Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. Society for Industrial and Applied Mathematics, 2nd edition, 2008.
- [7] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, 2nd edition, 2002.
- [8] Suseong Kim. ND-Aligner: A neural discriminative forced aligner, 2026. Manuscript.
- [9] John Lafferty, Andrew McCallum, and Fernando C. N. Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of the 18th International Conference on Machine Learning (ICML)*, pages 282–289, 2001.
- [10] Lawrence R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.
- [11] Kevin J. Shih, Rafael Valle, Rohan Badlani, Adrian Lańcucki, Wei Ping, and Bryan Catanzaro. RAD-TTS: Parallel flow-based TTS with robust alignment learning and diverse synthesis. In *ICML Workshop on Invertible Neural Networks, Normalizing Flows, and Explicit Likelihood Models*, 2021.
- [12] Charles Sutton and Andrew McCallum. An introduction to conditional random fields. *Foundations and Trends in Machine Learning*, 4(4):267–373, 2012.